AI Resume Analyzer

COMPLETE PROJECT GUIDE

Hire smarter with AI-powered resume intelligence.

Everything about the project in one place: what it does, how every feature and the AI work, the architecture and technology, and how to present it to anyone.

Java 21 · Spring Boot 3 | React 18 · TypeScript | Google Gemini AI | PostgreSQL + pgvector | GraphQL | Live on Render + Neon

Abhishek Raj
Developer · November 2026

ai-resume-analyzer-nwh6.onrender.com
github.com/Abhishek360007/Ai_Resume_Analyzer

OVERVIEW

Contents

01 Project at a glance
02 The problem and the solution
03 Who uses it: roles and permissions
04 The complete workflow
05 Feature tour with screenshots
06 How the AI works
07 System architecture
08 The resume processing pipeline
09 Technology stack
10 Database design
11 APIs
12 Security
13 Reliability and engineering decisions
14 Testing and quality
15 Deployment and operations
16 How to present the project
17 Questions you will be asked
18 Future roadmap
19 Glossary
20 Credits

How to use this guide. Read sections 1–5 to understand *what* the product does. Read 6–15 to understand *how* it is built. Use 16–17 before any demo, presentation or interview.

01 · PROJECT AT A GLANCE

A recruiting assistant powered by AI

AI Resume Analyzer is a full-stack web application that reads resumes with AI, turns them into structured candidate profiles, and ranks every candidate against a job requirement, with a score and a clear explanation for each one.

> "Upload a batch of resumes. The AI reads each one, pulls out skills, experience and education, then ranks every candidate against your job, explaining *why* each person scored the way they did. A human stays in control of every decision."

Key numbers

4
user roles with different access

94
GraphQL operations (53 queries, 41 mutations)

16
database tables, managed by Flyway

400+
automated tests (backend, frontend, browser)

4
score dimensions per match

768
numbers per text embedding (vector size)

4
AI providers supported (Gemini, OpenAI, Ollama, LM Studio)

$0
monthly hosting cost (free tiers)

What it can do

1

Read resumes

PDF, Word (DOC/DOCX), plain text and ZIP archives of many resumes, up to 50 MB per file.

2

Extract profiles

Name, contact, skills, years of experience, education, domain knowledge, work history and social links.

3

Human review

Recruiters confirm or correct what the AI extracted before a candidate is used for matching.

4

Match and rank

Scores for skills, experience, education and domain, plus strengths, gaps and a recommendation.

5

Visual insights

Radar chart against the pool average, score distribution, and the pool's weakest area.

6

Share results

Shortlist, select, and export a polished PDF report for hiring managers.

Live links

Live website	https://ai-resume-analyzer-nwh6.onrender.com
Source code	https://github.com/Abhishek360007/Ai_Resume_Analyzer
AI provider	Google Gemini (`gemini-3.5-flash-lite` + `gemini-embedding-001`)

02 · THE PROBLEM AND THE SOLUTION

Why this project exists

The problem

Too many resumes

A single opening can attract hundreds of applications. Reading each one takes several minutes, so screening alone takes days.

Tired humans miss people

Manual screening is inconsistent. The 200th resume gets less attention than the 5th, and strong candidates are overlooked.

Keyword filters are blunt

Traditional applicant tracking systems match exact words. "Spring Boot" and "Spring Framework" look unrelated to a keyword search.

Decisions are hard to explain

When a filter rejects someone, nobody can say why. Hiring managers need reasons, not just a yes or no.

The solution

1. **The AI reads every resume like a person would**
   A large language model understands context, so it extracts structured data from resumes in any layout.
2. **Matching by meaning, not just keywords**
   Resume text is converted into embeddings (numbers that capture meaning), so related skills and experience are recognised even with different wording.
3. **Every score is explained**
   Each match has four sub-scores plus written strengths, gaps and a recommendation, so a recruiter can check and trust it.
4. **A human stays in control**
   Recruiters confirm the AI's extraction, and people (not the AI) shortlist and select candidates.

The impact

Before	With AI Resume Analyzer
Hours or days to screen a batch of resumes	Minutes: upload once, then the batch is processed in the background
Inconsistent, tiring manual screening	The same criteria applied to every candidate
Keyword-only filters	Semantic understanding of skills and experience
"Rejected" with no reason	Scores, strengths, gaps and a recommendation for each person
Results stuck in someone's inbox	Shared, role-based workspace and exportable PDF reports

03 · WHO USES IT

Roles and permissions

Every person logs in with their own account. What they see and can do depends on their role. This is enforced on the server, not just hidden in the interface.

A

Administrator

Full access: users, skills master list, system health, every recruiting feature.

R

Recruiter

Creates job requirements, uploads and confirms resumes, runs matching.

H

HR

Manages employee records and can view candidates and download match reports.

M

Hiring Manager

Reviews jobs, candidates and match results; shortlists, selects and gives feedback.

Permission matrix

Capability	Admin	Recruiter	HR	Hiring Manager
Dashboard	✓	✓	✓	✓
View candidates	✓	✓	✓	✓
Upload resumes and confirm candidates	✓	✓	—	—
Delete candidates	✓	—	—	—
View job requirements	✓	✓	—	✓
Create, edit and delete job requirements	✓	✓	—	—
Load or remove sample jobs	✓	—	—	—
Run AI matching	✓	✓	—	—
View match results, shortlist and select	✓	✓	—	✓
Export PDF match report	✓	✓	✓	✓
Give feedback on candidates and jobs	✓	✓	—	✓
Employee management	✓	—	✓	—
User management (accounts, passwords)	✓	—	—	—
Skills master list	✓	—	—	—
Admin dashboard (system health, queue, audits)	✓	—	—	—

Why it matters: in a real company, not everyone should change job postings or user accounts. Role-based access control (RBAC) keeps data safe and responsibilities clear.

04 · THE COMPLETE WORKFLOW

From a pile of resumes to a hiring decision

This is the journey a recruiter takes through the application. Each step maps to a page you can show in a demo.

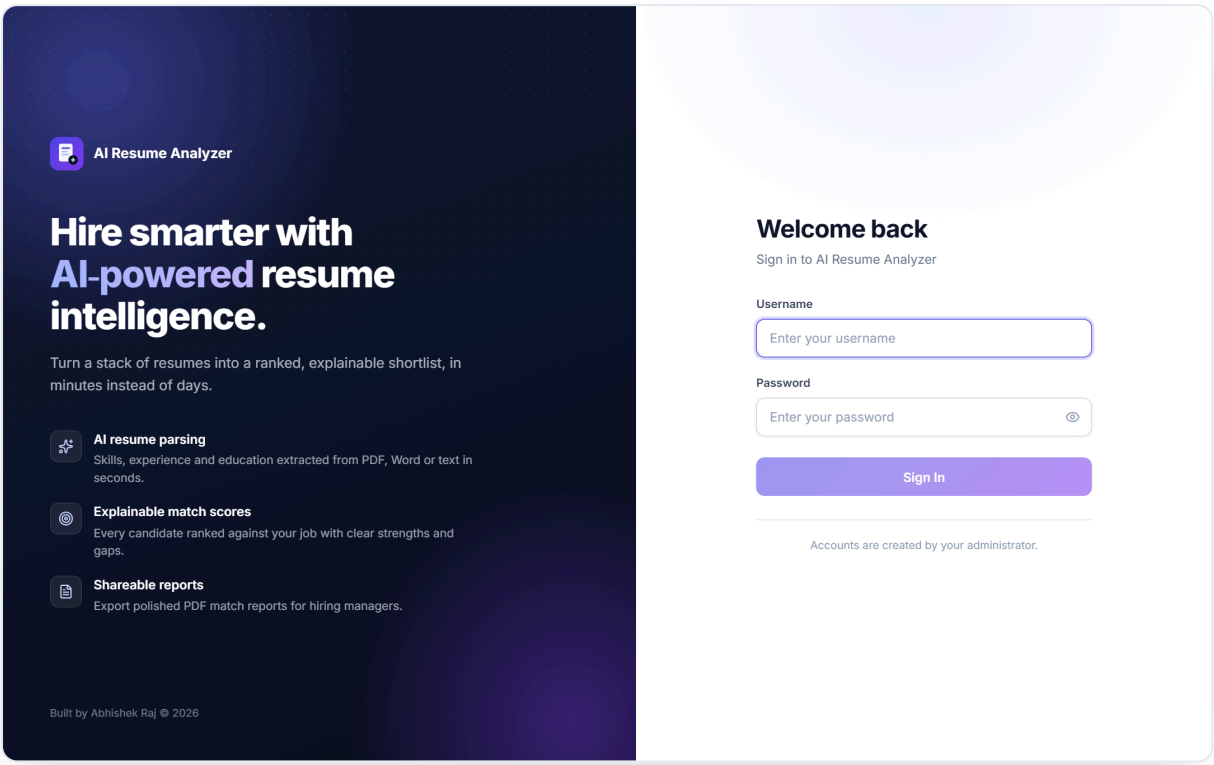
1. **Sign in**
   Each user logs in. A secure token (JWT) identifies them and their role for every request.
2. **Upload resumes**
   Drag and drop one or many files, or a ZIP. Each file becomes a job in a background queue, and progress updates live on screen.
3. **AI processing in the background**
   Text is extracted, split into sections, converted into embeddings, and analysed by the AI into a structured profile.
4. **Confirm candidates**
   New candidates wait on the Confirmations page with status `PENDING_CONFIRMATION`. The recruiter reviews or corrects the details and confirms; the candidate becomes `ACTIVE`.
5. **Create a job requirement**
   Title, description, required skills, experience range, education, domain and location.
6. **Match candidates**
   One click on **Match All Candidates** scores every active candidate against the job.
7. **Review results**
   A ranked list with four sub-scores per person, charts comparing them with the pool, strengths, gaps and recommendations. Scores of 70% or more are shortlisted automatically.
8. **Decide and share**
   Shortlist and select candidates, add feedback, and export a PDF report for the hiring manager.
9. **Monitor**
   Admins watch system health, the processing queue and the history of every matching run.

05 · FEATURE TOUR

The product, screen by screen

Login

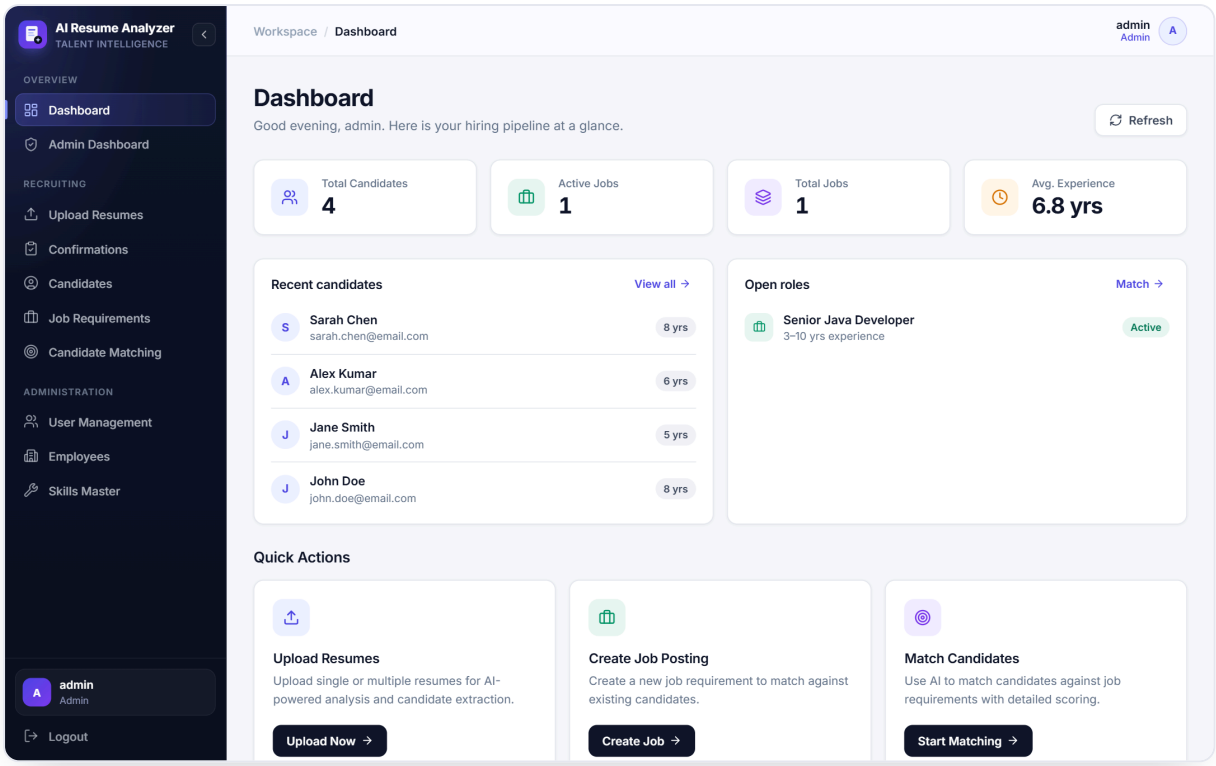
A split-screen sign-in page: product highlights on the left, the login form on the right, with a show-password toggle and clear error messages. Accounts are created by an administrator; there is no public sign-up.



Login page. Sessions use JSON Web Tokens that expire automatically; an expired session returns the user here with a friendly message.

Dashboard

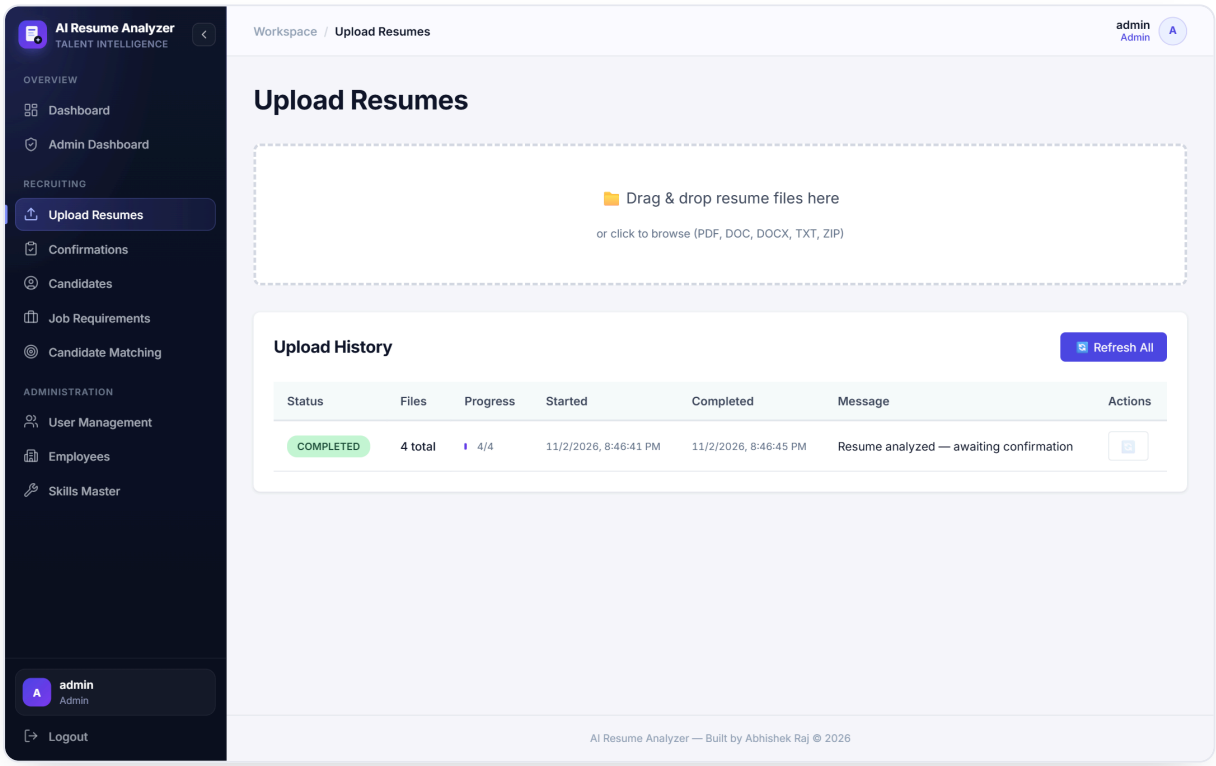
The home page for every user. It greets them by name and shows the hiring pipeline at a glance: total candidates, active jobs, total jobs and average experience, followed by recent candidates, open roles and quick actions.



Dashboard. The sidebar groups pages into Overview, Recruiting and Administration, and only shows what the user's role allows.

Upload Resumes

Drag and drop files, or click to browse. Supported: `.pdf`, `.doc`, `.docx`, `.txt` and `.zip` (a ZIP is expanded into one job per resume). The Upload History table updates **live** through Server-Sent Events, showing each batch's status, progress, timings and messages.



Upload Resumes. If the AI provider fails, the message column shows the real reason (for example a rate limit or invalid key), and the job is retried automatically.

Confirmations

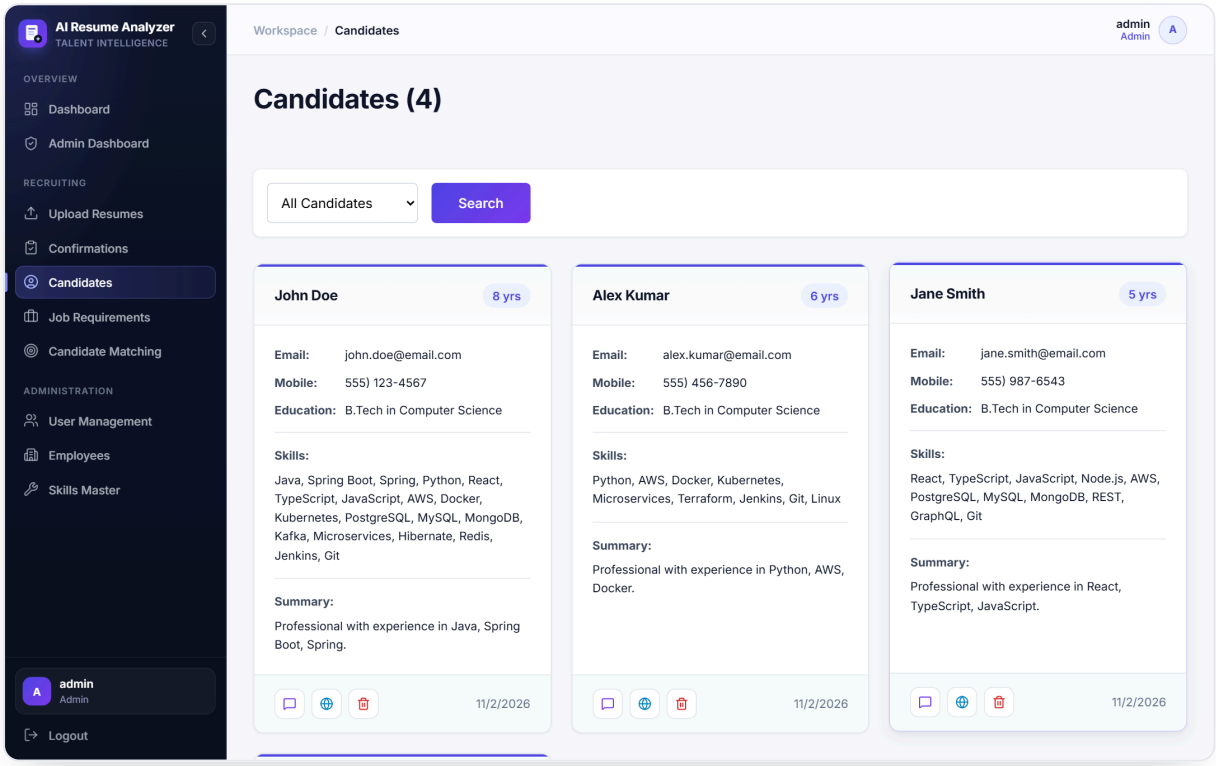
Every AI-extracted candidate first appears here. The recruiter checks name, email, phone, skills, experience and education against the original resume, edits anything that is wrong, and confirms. Only confirmed (`ACTIVE`) candidates take part in matching. This "human in the loop" step protects quality and fairness.

Candidates

All confirmed candidates as cards: contact details, education, skills and an experience summary. From each card you can:

- **Give feedback:** ratings and comments from the team.
- **View external profiles:** optional enrichment from GitHub, LinkedIn, Twitter/X or a web search, when API keys are configured.
- **Delete** a candidate (admins only, with confirmation); related matches and data are removed too.

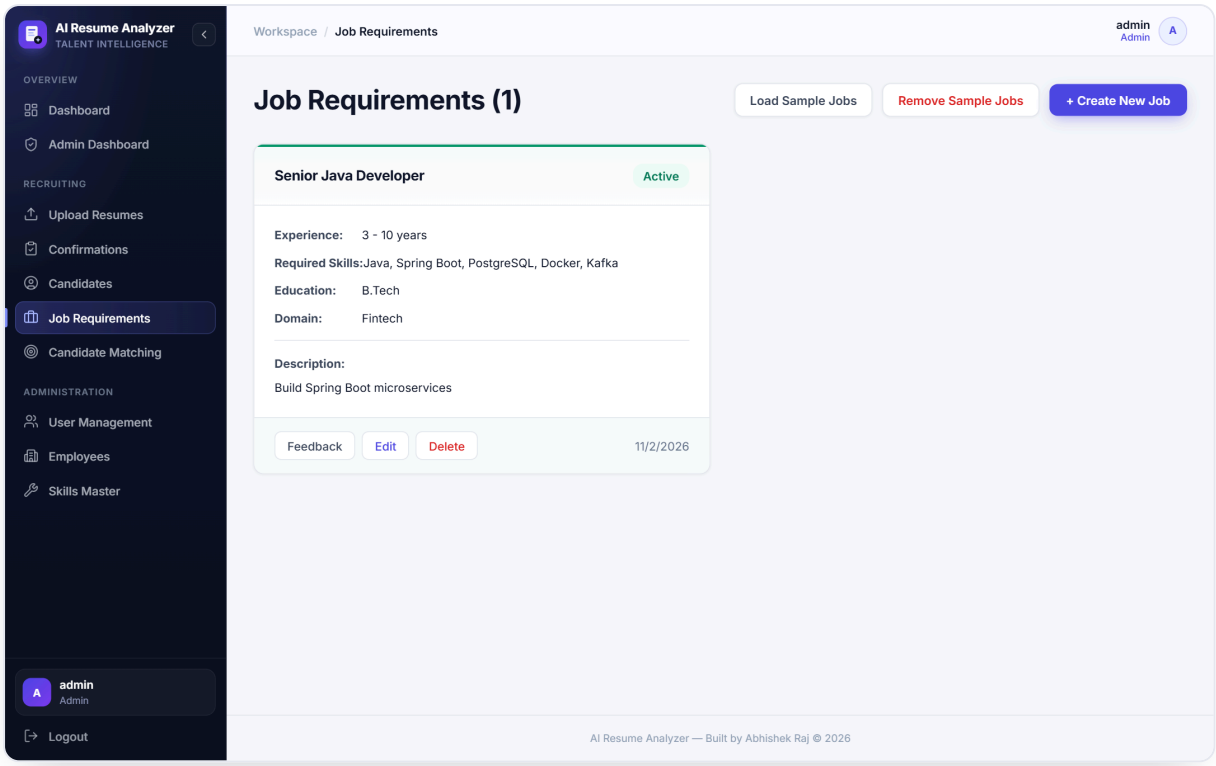
Candidates can be filtered and searched by name, skill or experience range.



Candidates. Profiles are built entirely from the AI's extraction, confirmed by a recruiter.

Job Requirements

Recruiters create and manage openings: title, description, required skills (linked to the skills master list), minimum and maximum experience, education, domain and location. Jobs can be edited, deactivated or deleted, and receive feedback. Admins can use **Load Sample Jobs** to fill the system with realistic examples for demos.

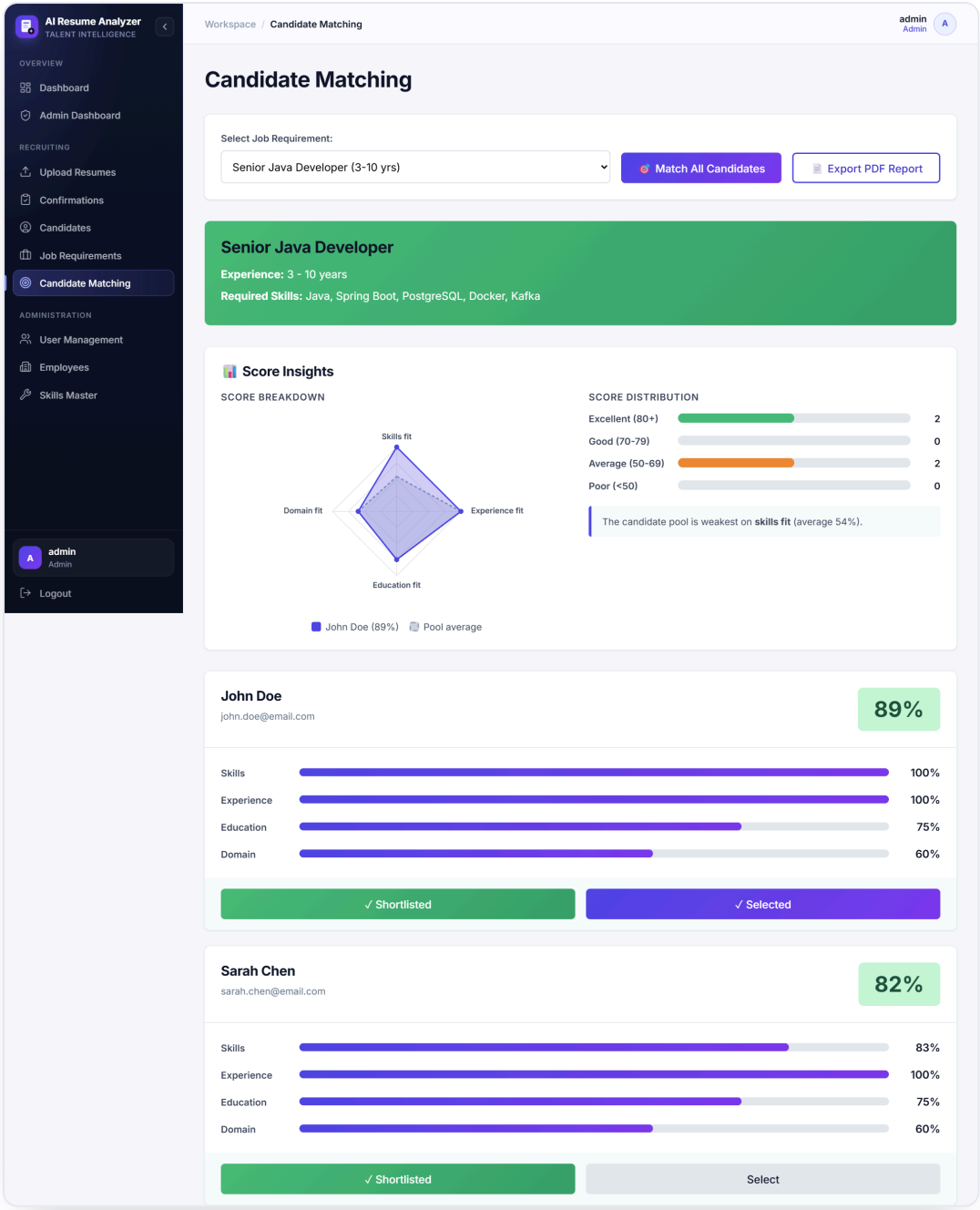


Job Requirements. Each card shows the criteria the AI uses when scoring candidates.

Candidate Matching: the heart of the product

Select a job and click **Match All Candidates**. Every active candidate is scored, and the page shows:

- **Score Insights:** a radar chart comparing the top candidate with the pool average across skills, experience, education and domain; a score distribution (Excellent 80+, Good 70–79, Average 50–69, Poor below 50); and a callout naming the pool's weakest area.
- **Ranked candidate cards** with the overall match percentage and four score bars.
- **Shortlist** and **Select** buttons to record decisions (70%+ is shortlisted automatically).
- **Export PDF Report:** a downloadable report of the ranked results for the hiring manager.



Candidate Matching. John Doe scores 89% (skills 100%, experience 100%, education 75%, domain 60%) for a Senior Java Developer role.

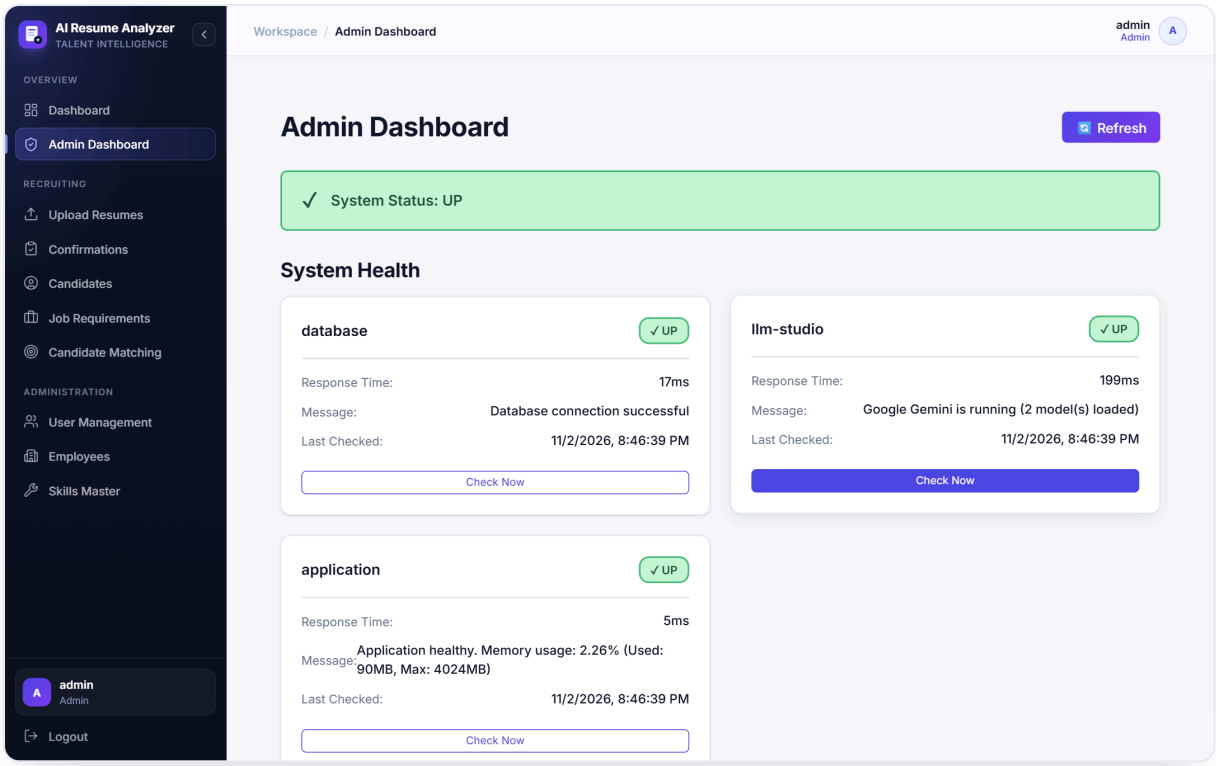
PDF match report

One click produces a PDF, generated on the server with Apache PDFBox, containing the job's details and every matched candidate ranked by score, with the sub-scores and the AI's explanation. It is ready to email or print for people who don't use the app.

Admin Dashboard

The control room for administrators:

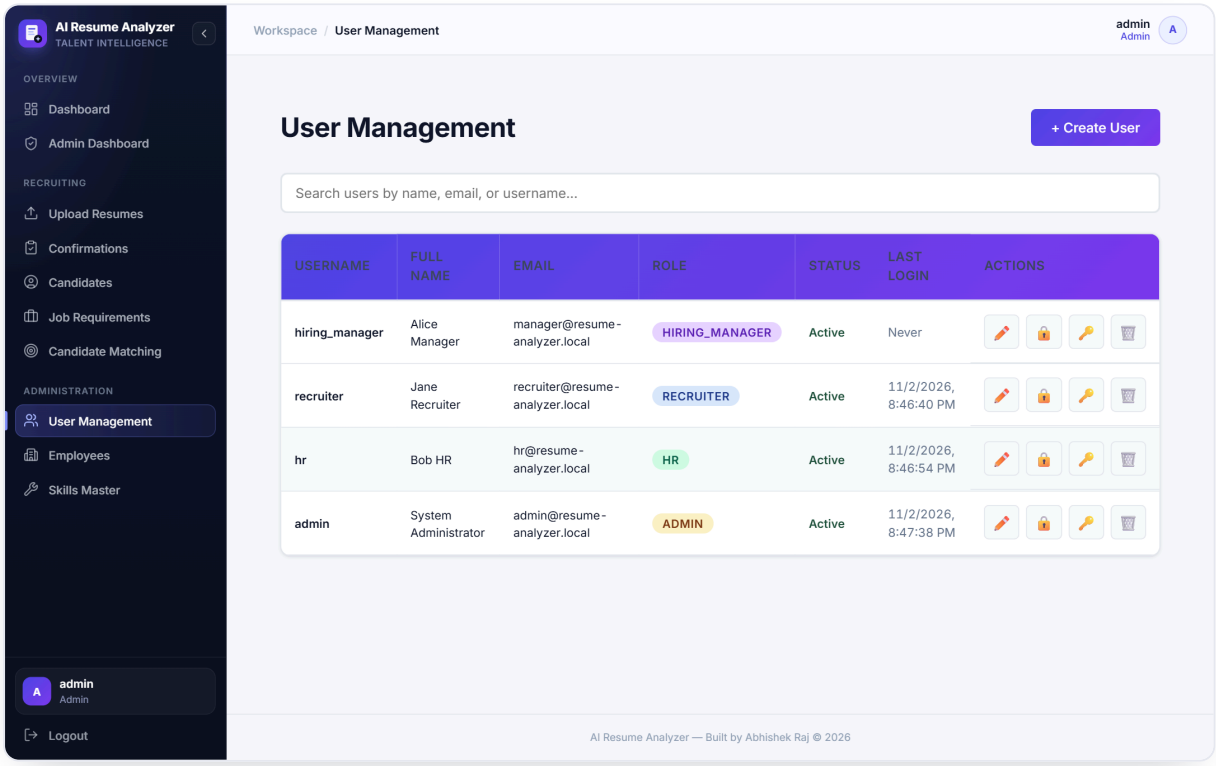
- **System health:** live checks of the database, the AI provider (it reports how many models are available, or the exact problem, such as an invalid key) and the application's memory use. Checks run every 5 minutes, or on demand with **Check Now**.
- **Resume pipeline:** active workers, queue depth, average processing time and job counts by status.
- **User and employee statistics** by role, department and employment type.
- **Candidate match runs:** an audit history of every matching run (job, status, candidates, shortlisted, average and top score, duration, estimated AI tokens and who started it).



Admin Dashboard. Problems are visible before a user reports them.

User Management

Administrators create accounts, assign roles, activate or deactivate users, reset passwords and search the user list. Every login and password change is written to an audit log.



User Management. Only administrators can see this page, and the server rejects anyone else's request.

Skills Master, Employees and Feedback

Skills Master

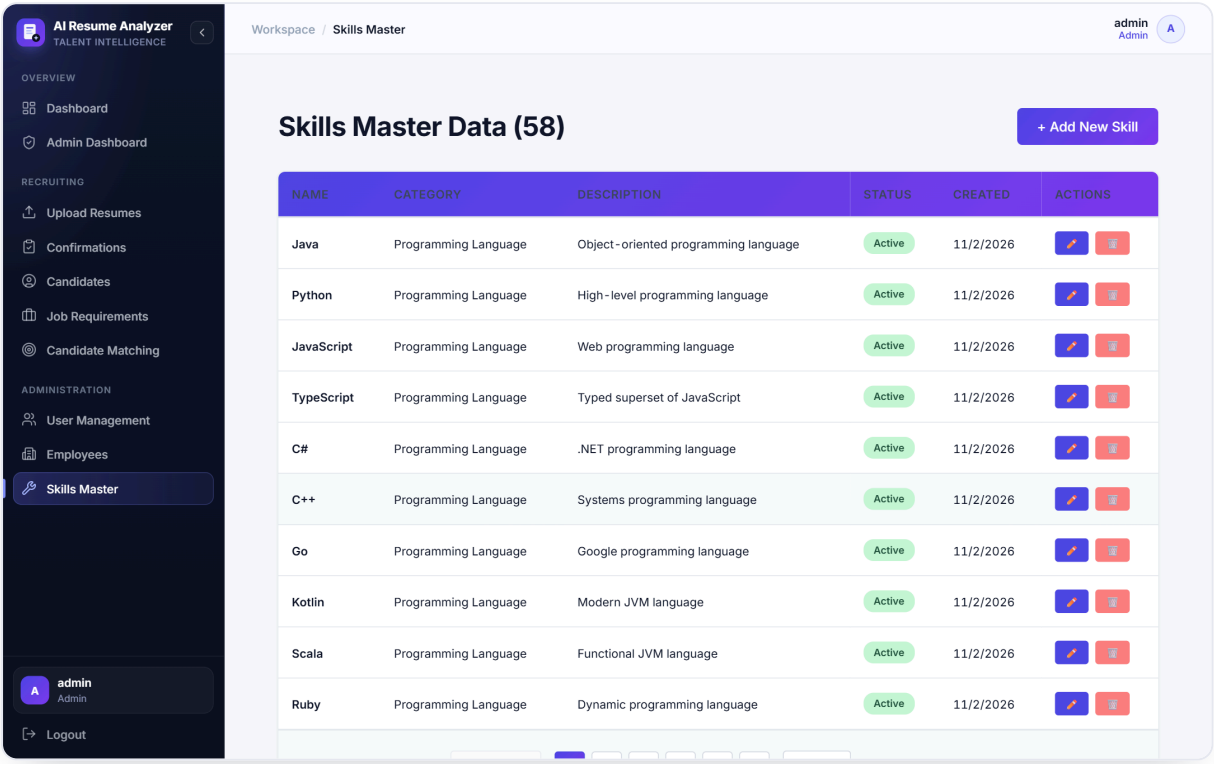
A curated list of skills with categories, used when defining job requirements so skills are named consistently.

Employees

HR and admins manage employee records: department, employment type, status (active, on leave, suspended, terminated) and reporting manager.

Feedback

Recruiters, hiring managers and admins rate and comment on candidates and jobs; averages are calculated automatically, and feedback can be hidden.



Skills Master. Skills can be added, edited, deactivated and grouped by category.

06 · HOW THE AI WORKS

Inside the intelligence

The application uses two different kinds of AI model: a **language model** that reads and reasons, and an **embedding model** that turns text into numbers capturing its meaning.

Step 1: Resume analysis (extraction)

After the text is extracted from the file, the language model receives it with carefully written instructions (prompts kept in `ai-prompts.yml`) and must return strict JSON with these fields:

Field	Meaning
name, email, mobile	Contact details
skills	Technical and professional skills
yearsOfExperience	Total professional experience in years
experienceSummary	A short professional summary
academicBackground	Degrees and institutions
domainKnowledge	Industries and business domains
workHistory	Employers with roles and years
linkedInUrl, githubUrl, twitterUrl	Social profiles, used for optional enrichment
confidenceScore	How confident the AI is in its extraction

If the AI is unavailable, the app never stores a fake "empty" candidate. It reports the real error and retries the job later.

Step 2: Embeddings and semantic search

The resume is split into sections (such as experience, skills and education). Each chunk is sent to the embedding model, which returns a list of **768 numbers** describing its meaning. These vectors are stored in PostgreSQL using the **pgvector** extension.

> **Simple explanation:** texts with similar meanings get similar numbers. "Built REST APIs with Spring Boot" and "Developed Java microservices" end up close together even though they share few words. Measuring this closeness is called *cosine similarity*.

Step 3: Matching with RAG (Retrieval-Augmented Generation)

When a candidate is matched to a job:

A
Embed the job
The job's title, description, skills and domain become a query vector.

→

B
Retrieve
pgvector finds the 5 resume sections most similar to the job.

→

C
Augment
Those sections are added to the candidate's profile and the job criteria.

→

D
Generate
The language model scores the match and explains it as JSON.

Retrieval gives the AI the most relevant evidence from the resume instead of everything, which improves accuracy and keeps requests small.

Step 4: The scoring model

The AI returns four sub-scores from 0 to 100, and the overall score is their weighted average:



It also returns written **explanation**, **strengths**, **gaps** and a **recommendation**. Candidates scoring **70% or more** are shortlisted automatically.

> **Worked example:** skills 100, experience 100, education 75, domain 60 → 0.40×100 + 0.25×100 + 0.20×75 + 0.15×60 = 40 + 25 + 15 + 9 = **89%**.

Step 5: Agentic enrichment (optional)

The matcher can act like a small agent that gathers more evidence before deciding:

- It refreshes stale external profiles (GitHub, LinkedIn, Twitter/X, web search) and builds a job-aware summary of them.
- **Multi-pass matching:** if a candidate's first score is **borderline (50–75%)** and no extra context was available, it enriches the profile and scores again.
- **Company verification:** employers listed in the work history can be researched and labelled `GENUINE`, `SUSPICIOUS` or `UNKNOWN`, and that impression is shared with the matcher.

These features use extra API keys (GitHub, Twitter, Tavily web search) and switch themselves off gracefully when keys are missing.

Choosing an AI provider

All AI calls go through Spring AI's OpenAI-compatible client, so the provider is chosen by a **configuration profile**, with no code change:

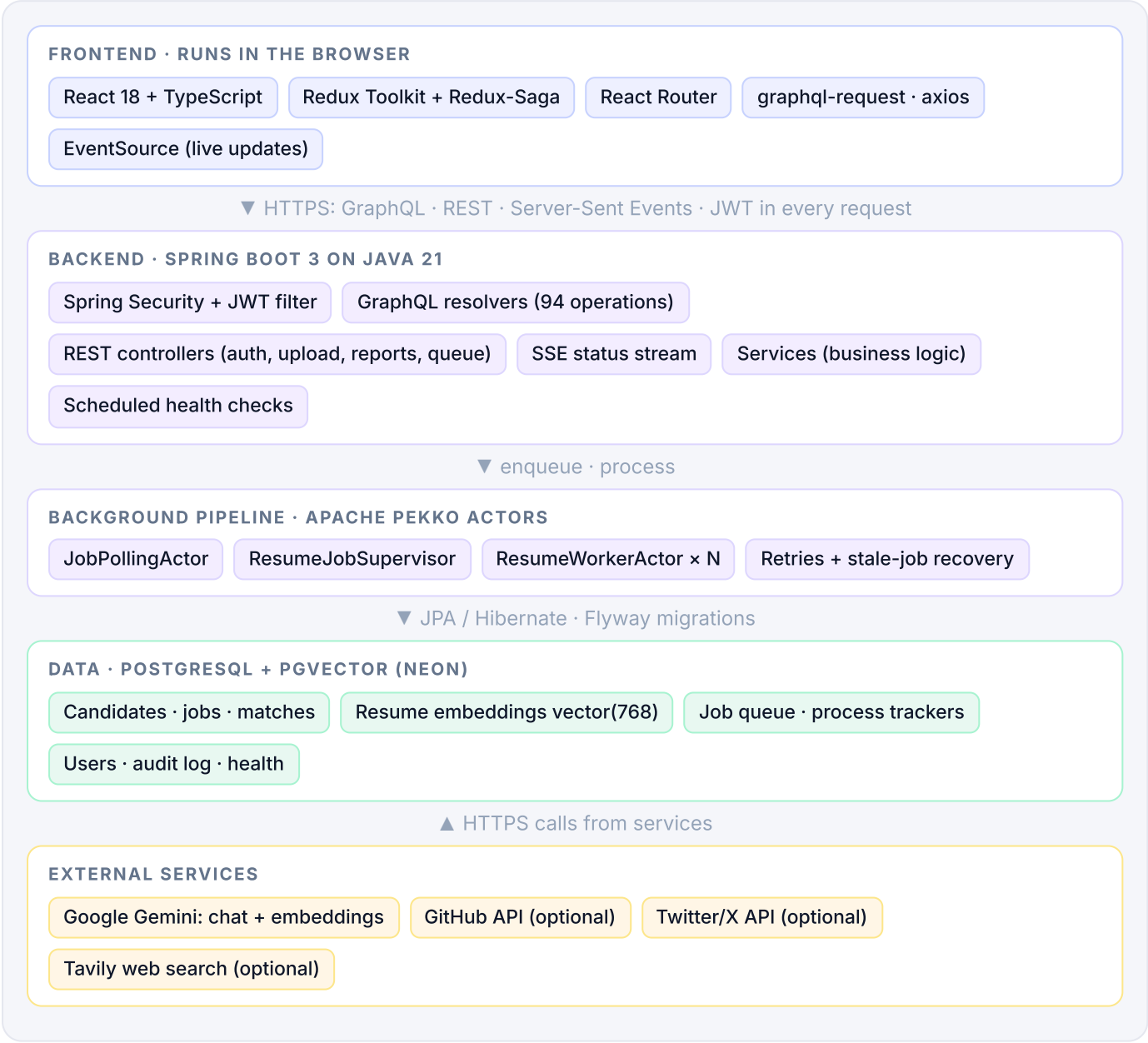
Profile	Provider	Best for
gemini	Google Gemini (free tier)	The live deployment: free and capable
openai	OpenAI (gpt-4o-mini, text-embedding-3-small)	Paid production use
ollama	Ollama, running models on your own computer	Privacy: data never leaves the machine
(default)	LM Studio, a local server	Offline development

> **Privacy note:** on Gemini's free tier, Google may use submitted content to improve its products, so demos use sample resumes only. Real candidate data should use a paid plan or a local model.

07 · SYSTEM ARCHITECTURE

How the pieces fit together

A single Spring Boot application serves both the React frontend and the API. Long-running work happens in a background pipeline so the interface stays fast.



Design principles

One deployable unit

The React build is bundled into the Spring Boot JAR and packaged in one Docker image, which makes deployment simple.

Asynchronous by design

Uploads return immediately; AI work happens in the queue and progress is pushed to the browser.

Provider-agnostic AI

The AI provider is an interchangeable configuration choice, not a hard dependency.

Database as the source of truth

The job queue lives in PostgreSQL, so queued work survives restarts and crashes.

08 · THE RESUME PROCESSING PIPELINE

What happens after you click upload

STAGE 1
INITIATED
Files saved, a tracker created, one queue job per file.

→

STAGE 2
EMBED_GENERATED
Text extracted, split into sections, embeddings created.

→

STAGE 3
VECTOR_DB_UPDATED
Vectors stored in pgvector.

→

STAGE 4
RESUME_ANALYZED
AI extracts the structured profile.

→

STAGE 5
COMPLETED
Candidate saved for confirmation.

Step by step

1. **Upload:** `POST /api/upload/resume` validates file types, stores the files, creates a *process tracker* for the batch and adds one row per file to the `job_queue` table. The response returns immediately.
2. **Polling:** the `JobPollingActor` regularly claims pending jobs from the database and hands them to the `ResumeJobSupervisor`.
3. **Processing:** a pool of `ResumeWorkerActor`s processes jobs in parallel (2 workers on the free server). Each job runs in a database transaction.
4. **Text extraction:** Apache PDFBox for PDF, Apache POI for Word, and UTF-8/Windows-1252 decoding for text files.
5. **Embeddings and analysis:** as described in section 6.
6. **Live progress:** every stage change is published through Server-Sent Events to `/api/upload/status/events`, so the upload table updates without refreshing.

When things go wrong

Situation	What the system does
AI provider rate limit (HTTP 429) or overload (503)	Waits and retries up to 4 times with increasing delays (5 s, 10 s, 20 s)
A job fails	Marks it for retry (up to 3 attempts) and shows the real reason in the upload history
A worker stops responding	Stale jobs are detected and re-queued; after the last attempt the batch shows a clear "failed permanently" message
Server restarts mid-batch	Pending jobs are still in the database and are picked up again
Unsupported or broken file	That file fails with a readable message; the rest of the batch continues

09 · TECHNOLOGY STACK

Tools and why they were chosen

Technology	Layer	Why it is used
Java 21	Language	Modern long-term-support Java with records, pattern matching and strong performance
Spring Boot 3.2	Backend framework	Industry standard for production web services: configuration, dependency injection, monitoring
Spring AI	AI integration	One abstraction for chat and embedding models across providers, with retry support
Spring for GraphQL	API	Clients request exactly the fields they need, from one typed schema
Spring Security + JWT	Security	Stateless token authentication and method-level role checks
Spring Data JPA / Hibernate	Data access	Maps Java entities to database tables
Flyway	Database migrations	Versioned SQL scripts (V1–V5) create and evolve the schema automatically
PostgreSQL 16+	Database	Reliable relational database
pgvector	Vector search	Stores embeddings and runs similarity search inside PostgreSQL, so no separate vector database is needed
Apache Pekko	Concurrency	Actor model for a fault-tolerant background processing pipeline
Apache PDFBox / POI	Documents	Read PDF and Word resumes; generate PDF reports
React 18 + TypeScript	Frontend	Component-based UI with type safety
Redux Toolkit + Redux-Saga	State	Predictable application state and side effects (API calls, login flow)
Vite	Build tool	Fast development server and optimised production builds
lucide-react	Icons	Consistent, lightweight icon set
JUnit 5, Mockito	Backend tests	Unit and integration tests
Vitest, Testing Library	Frontend tests	Component and logic tests
Playwright	End-to-end tests	Drives a real browser through complete user journeys for every role
Maven	Build	Builds the frontend and backend into one JAR
Docker	Packaging	One image that runs the same everywhere
Render	Hosting	Free Docker hosting, deployed from GitHub
Neon	Managed database	Free serverless PostgreSQL with pgvector that does not expire

10 · DATABASE DESIGN

Where the data lives

Sixteen tables, created and versioned by five Flyway migrations. The `vector` extension is enabled in the first migration.

Table	Purpose
candidates	Candidate profiles extracted by the AI, their status (pending or active) and the original resume file
resume_embeddings	Resume sections and their `vector(768)` embeddings for semantic search
job_requirements	Job openings and their criteria
job_requirement_skills	Links jobs to skills in the master list
skills	The skills master list, with categories
candidate_matches	Each candidate–job match: overall and sub-scores, explanation, strengths, gaps, recommendation, shortlisted and selected flags
match_audits	History of matching runs: who started it, status, counts, scores, duration and estimated tokens
job_queue	Background jobs: status, priority, attempts, heartbeat and errors
process_tracker	Progress of each upload batch: total, processed and failed files, stage and message
candidate_external_profiles	Enriched data from GitHub, LinkedIn, Twitter/X and web search
company_impressions	Research verdicts on companies in a candidate's work history
feedback	Ratings and comments on candidates and jobs
users	Accounts, roles and BCrypt password hashes
employees	Employee records managed by HR
audit_log	Security events: logins, failures, password changes and account changes
system_health	Latest health status of the database, AI provider and application

Key relationships

- A **candidate** has many **resume embeddings**, **matches**, **external profiles** and **company impressions**; deleting a candidate removes them too (cascade).
- A **job requirement** has many **matches** and **match audits**, and many **skills** through the link table.
- An **upload batch** (process tracker) has one **queue job** per file.

How the frontend talks to the backend

GraphQL: one endpoint, 94 operations

Most data flows through `POST /graphql`. Examples by area:

Area	Queries	Mutations
Candidates	allCandidates, candidate, searchCandidatesByName, searchCandidatesBySkill, candidatesByExperienceRange, pendingCandidates	confirmCandidate, discardCandidate, deleteCandidate
Jobs	allJobRequirements, activeJobRequirements, searchJobsByTitle	create / update / deactivate / delete JobRequirement, load / remove sample jobs
Matching	matchesForJob, topMatchesForJob, shortlistedCandidatesForJob, selectedCandidatesForJob	matchAllCandidatesToJob, matchCandidateToJob, matchCandidateToAllJobs, updateCandidateMatch
Skills	allSkills, activeSkills, searchSkills, skillCategories	createSkill, updateSkill, deleteSkill
Users	me, allUsers, usersByRole, userStatistics	createUser, updateUser, activate / deactivate, resetUserPassword
Employees	allEmployees, employeesByDepartment, employeeStatistics	createEmployee, updateEmployee, terminateEmployee
Feedback	feedbackForCandidate, feedbackForJob, averageRating…	submitFeedback, updateFeedback, hide / show
Monitoring	systemHealth, overallSystemStatus, matchAudits, activeMatchRuns, allProcessTrackers	checkServiceHealth, resetFailureCount
Enrichment	candidateExternalProfiles, companyImpressionsForCandidate	enrichCandidateProfile, enrichCandidateProfileFromUrl

REST endpoints

Endpoint	Purpose
POST /api/auth/login · /refresh · /logout	Sign in, renew and end a session
POST /api/auth/change-password	A user changes their own password
POST /api/upload/resume	Upload one or many resumes (multipart)
GET /api/upload/status/events	Live upload progress (Server-Sent Events)
GET /api/reports/jobs/{id}/matches.pdf	Download the PDF match report
GET /api/jobs/health · /stats · /queue/depth	Queue monitoring (admin only)
GET /actuator/health	Health check used by the hosting platform

12 · SECURITY

Keeping data and access safe

Authentication with JWT

After login the server issues a signed JSON Web Token (plus a refresh token). Every request carries it; the server verifies the signature and expiry. The signing secret is generated by the hosting platform.

Password storage

Passwords are never stored, only BCrypt hashes, which are slow to crack by design.

Role checks on the server

Every GraphQL operation and admin endpoint has a `@PreAuthorize` rule. Hiding a button is not enough; the server itself refuses.

A test that guards security

An automated test scans every GraphQL query and mutation and **fails the build** if any lacks a permission rule.

Safe error messages

Users see readable business errors; internal details and stack traces stay in the server logs.

Audit trail

Logins, failed attempts, password changes and account changes are recorded with time, IP address and browser.

Secrets out of code

API keys and database passwords live in environment variables on the host, never in the repository.

Upload validation

Only allowed file types are accepted, with size limits (50 MB per file, 100 MB per request).

Real problems found and solved

These are real issues found while taking the project to production. They make excellent talking points because they show debugging skill, not just feature building.

Problem	Root cause	Solution
Uploads failed on Gemini with a crash in the AI library	Gemini's embedding response has no "usage" block, which made Spring AI throw a NullPointerException	A lightweight direct embedding client is used as a fallback, so embeddings work with any provider
Embeddings of the wrong size	Gemini returns 3072-number vectors, but the column holds 768	Request 768 dimensions, and safely truncate and re-normalise longer vectors
"Match All" failed on the free AI tier	Rate limits (HTTP 429) were not retried by default	Retries on 429/503 with exponential back-off (tested against a simulated rate limiter)
A retired AI model broke analysis	Google stopped offering the old model to new users	Moved to a current model; the model name is configurable through `GEMINI_MODEL`
Logins failed while uploads were running	Each live-update stream held a database connection open	Excluded the streaming endpoint from open-session-in-view; tested with 20 open streams
Jobs stuck in "processing" forever	A failed transaction also rolled back the "will retry" bookkeeping	Restructured transactions so failure handling is committed separately
Progress showed 1/4 when all 4 were done	Parallel workers overwrote each other's counter updates	Atomic SQL updates for progress counters
Very slow start on the small free server	The Java compiler competed with startup for 0.1 CPU	Tuned JVM flags for small containers (memory limits, quick-compiler only)
Admin "reset password" did nothing	The operation returned success without saving	Implemented the reset properly, with validation and a test
Free database would expire after 30 days	Render's free PostgreSQL is temporary	Migrated the data to Neon in the same region as the server

14 · TESTING AND QUALITY

Proof that it works

201	106	103
backend tests (JUnit 5 + Mockito)	frontend unit tests (Vitest)	end-to-end browser tests (Playwright)

What is tested

- **Services:** resume parsing for every file type, AI response handling and fallbacks, embedding sizes, matching and auto-shortlisting, job queue retries and stale-job recovery.
- **Security:** every GraphQL operation must declare a permission rule; error messages must not leak internals.
- **Frontend:** components, state slices, sagas and error-message handling.
- **End to end:** real browser sessions for every role (admin, recruiter, HR, hiring manager), checking navigation, permissions, job creation, uploads and matching.
- **Full-flow check:** a 14-step script uploads four resumes, confirms them, creates a job, matches, shortlists, exports the PDF and verifies the audit, against a simulated AI server that can also simulate rate limits.

> **Why a simulated AI server?** It makes tests fast, free and repeatable, and lets us test failure cases (rate limits, missing fields, outages) that are hard to trigger with the real provider.

15 · DEPLOYMENT AND OPERATIONS

How it runs on the internet

1
GitHub
Code is pushed to the main branch.

→

2
Render build
The Dockerfile builds the frontend and backend into one image.

→

3
Health check
The new version only goes live once `/actuator/health` reports UP.

→

4
Live
Served over HTTPS; Flyway updates the database on start.

The whole setup is described in `render.yaml` (infrastructure as code), so it can be recreated with one click.

Configuration (environment variables)

Variable	Purpose
SPRING_PROFILES_ACTIVE	Selects the AI provider profile (gemini)
GEMINI_API_KEY	Key for Google Gemini (kept secret)
GEMINI_MODEL	Optional: switch the AI model without a code change
DB_HOST · DB_NAME · DB_USERNAME · DB_PASSWORD	Connection to the Neon database
DB_SSL_MODE	`require`: encrypted database connection
JWT_SECRET	Signing key for login tokens, generated by Render
JAVA_OPTS	JVM tuning for the small free server
PEKKO_WORKERS	Number of parallel resume workers (2)

Free-tier behaviour to know

- **Sleeping server:** Render's free service sleeps after 15 minutes without visitors and takes a few minutes to wake. Open the site before a demo.
- **Database:** Neon pauses an idle database after 5 minutes and wakes in about a second; the connection pool is tuned for this.
- **AI limits:** Gemini's free tier limits requests per minute and per day. Keep live batches small (5 candidates or fewer).

Running it locally

Start PostgreSQL with pgvector (for example the `pgvector/pgvector:pg16` Docker image), set the database and AI variables, then run `mvn clean package` and `java -jar target/resume-analyzer-1.0.0.jar`. The app is served at `http://localhost:8080`.

16 · HOW TO PRESENT THE PROJECT

Your presentation playbook

The 30-second pitch (learn it by heart)

> "Recruiters spend hours reading resumes for every opening. AI Resume Analyzer does that first pass in minutes. You upload a batch of resumes, the AI pulls out each candidate's skills, experience and education, then ranks every candidate against a job with a score and a clear reason. It's live on the web, secure with role-based logins, tested with over 400 automated tests, and runs at zero hosting cost."

Structure for a 5–10 minute talk

1. **Problem (1 min):** the pain of manual screening (section 2).
2. **Solution (30 s):** the pitch above.
3. **Live demo (5 min):** the script on the next page.
4. **How it works (2 min):** the pipeline and RAG matching (sections 6–8).
5. **Engineering (1 min):** two or three stories from section 13.
6. **Close (30 s):** impact and what comes next.

Adjust to your audience

Audience	Focus on	Keep short
Client, HR, business	Time saved, explainable scores, human confirmation, PDF reports	Technology details
Senior engineer, interviewer	Architecture, queue and retries, RAG, security test, debugging stories	The basic feature tour
Friends, students	The story and the live demo	Internals
Judges, events	Problem → demo → architecture → impact in about 5 minutes	Long tangents

Before every demo

1. Open the site **5–10 minutes early** so the server is awake.
2. Log in as admin → Admin Dashboard → **Check Now**; all services should be UP.
3. Have 2–3 sample resumes on your desktop and one job already created.
4. Keep live matching to 5 candidates or fewer.
5. Backup plan: the screenshots in this guide, if the internet fails.
6. Never show passwords on screen.

Live demo script (about 7 minutes)

1 · Login — 15 s

Sign in as admin.

"It's a secure web app: each team member has their own login and role."

2 · Dashboard 30 s

Point at the stat cards, recent candidates and open roles.

"This is the hiring pipeline at a glance."

3 · Upload Resumes 1 min

Drag in 2–3 sample resumes and watch the status update.

"I can drop in many resumes at once, even a ZIP. The status updates live; I don't refresh anything."

4 · Confirmations 45 s

Open a pending candidate and confirm it.

"The AI read the resume and extracted the profile. A recruiter quickly checks it, so a human stays in control."

5 · Candidates and Jobs 45 s

Show the candidate cards, then a job requirement.

"Each resume becomes a clean profile. Here I describe the role I'm hiring for."

6 · Candidate Matching ⭐ 2 min

Select the job → Match All Candidates. Slow down here: this is the "wow" moment.

"Everyone is ranked, and it explains why: this person matches Java and Spring but is missing Kafka. The radar chart compares the top candidate with the whole pool."

7 · Export PDF 20 s

Download the report.

"One click gives a report I can send to the hiring manager."

8 · Admin Dashboard 45 s

Show system health, the queue and the match-run history.

"Behind the scenes there's monitoring: the AI's status, the processing queue and an audit of every run."

9 · Role-based access 30 s

Log in as recruiter and show the shorter menu.

"Different roles see different things; the server enforces it too."

Closing line: "In short: resumes in, a ranked and explained shortlist out, in minutes. It's live, secure and tested, and it runs at zero hosting cost. Happy to go deeper into anything."

17 · QUESTIONS YOU WILL BE ASKED

Prepared answers

Can the AI be biased against candidates?

That's why the AI only recommends. Every score is explained, a recruiter confirms the extracted data, and people make the final choice. A natural next step is anonymised screening that hides names before scoring.

How accurate is it?

Extraction is reviewed by a human on the Confirmations page, and every match comes with sub-scores, strengths and gaps that can be checked against the resume. Retrieval (RAG) gives the AI the most relevant resume sections, which improves accuracy.

What about candidate privacy?

Data is stored in our own database with encrypted connections and role-based access. The demo uses sample resumes because free AI tiers may use data for training; real use would switch to a paid plan or a local model, which is only a configuration change.

What happens if the AI service is down or rate-limited?

Requests are retried with back-off; failed jobs stay in the queue and are retried; the exact reason appears in the upload history; and the Admin Dashboard shows the AI service status.

How does it compare with keyword-based tools?

Keyword tools match exact words. This system uses embeddings to compare meaning, and a language model that weighs skills, experience, education and domain, and explains its reasoning.

Why GraphQL instead of only REST?

The frontend has many views that need different slices of the same data. GraphQL lets each screen request exactly the fields it needs from one typed schema. REST is still used where it fits better: file uploads, live streams and PDF downloads.

Why actors (Pekko) for processing?

Resume processing is slow and can fail. Actors give a supervised pool of workers that process jobs in parallel and recover from failures, while the database-backed queue makes sure no job is lost on restart.

Why pgvector instead of a dedicated vector database?

It keeps vectors next to the rest of the data in one PostgreSQL database: simpler operations, transactions across both, and no extra service to pay for.

Does it scale?

The web layer is stateless (JWT), so more instances can be added; processing scales by increasing workers or instances that pull from the shared queue; and the database is a managed service that can be upgraded.

How much does it cost to run?

Nothing today: Render, Neon and Gemini all have free tiers. For real business use, roughly $7/month for an always-on server plus pay-as-you-go AI usage.

Did you build all of it yourself?

Be honest: "It started from an open-source Spring Boot project, which I extended into a production-ready application. I added Google Gemini support, PDF reports and score charts, the live cloud deployment and database migration, the reliability and security fixes, and the complete UI redesign." Honesty, with a clear list of what you did, earns more respect than claiming everything.

18 · FUTURE ROADMAP

Where it could go next

Anonymised screening
Hide names, photos and other personal details during scoring to reduce bias.

Job portal integration
Import applications directly from LinkedIn, Naukri or company career pages.

Notifications
Email or chat alerts when strong matches appear for an open role.

Interview workflow
Schedule interviews, collect interviewer scorecards and track hiring stages.

AI-generated interview questions
Create tailored questions from each candidate's gaps for a job.

Analytics
Time-to-hire, source quality and pipeline conversion reports.

Multi-company support
Separate workspaces for different organisations (multi-tenancy).

Candidate portal
Let applicants upload resumes and track their own status.

19 · GLOSSARY

Terms explained simply

LLM	Large Language Model: an AI trained on huge amounts of text that can read, write and reason (here, Google Gemini).
Embedding	A list of numbers representing the meaning of a text. Similar meanings produce similar numbers.
Vector	The list of numbers itself; here each has 768 numbers.
Cosine similarity	A measure of how closely two vectors point in the same direction, i.e. how similar two texts are in meaning.
pgvector	A PostgreSQL extension that stores vectors and finds the most similar ones quickly.
RAG	Retrieval-Augmented Generation: first retrieve the most relevant information, then give it to the AI to generate an answer.
Prompt	The instructions and data sent to the AI.
Token	A small piece of text (about ¾ of a word) that AI providers count for limits and billing.
Rate limit / 429	A provider refusing requests because too many arrived too quickly.
GraphQL	An API style where the client asks for exactly the data it needs from one endpoint.
REST	An API style based on URLs and HTTP methods such as GET and POST.
JWT	JSON Web Token: a signed token proving who the user is and what role they have.
RBAC	Role-Based Access Control: permissions depend on the user's role.
BCrypt	A deliberately slow password-hashing method that protects stored passwords.
SSE	Server-Sent Events: the server pushes live updates to the browser over one open connection.
Actor (Pekko)	An independent worker that processes messages one at a time; supervisors restart actors that fail.
Job queue	A list of pending work items processed in the background.
Flyway	A tool that applies versioned database changes automatically.
Docker	Packages the app and everything it needs into an image that runs the same everywhere.
Cold start	The delay while a sleeping server starts up again.
Spring profile	A named set of settings, used here to choose the AI provider.

20 · CREDITS

About this project

Developer	Abhishek Raj
Repository	github.com/Abhishek360007/Ai_Resume_Analyzer
Live site	ai-resume-analyzer-nwh6.onrender.com
License	MIT

Acknowledgement

This project builds on the open-source **spring-boot-ai-resume-analyzer** by d-subrahmanyam, released under the MIT License. It was extended and taken to production with:

- Google Gemini support, with fixes for its embedding API and model changes
- PDF match reports and the Score Insights charts
- Cloud deployment on Render and Neon, including a zero-downtime database migration
- Reliability work: rate-limit retries, transaction fixes, stale-job recovery, connection-pool fixes and cold-start tuning
- Security hardening: permission rules on every operation (with a test enforcing them), safe errors and a working password reset
- Plain-text resume support and readable error messages throughout
- A complete redesign of the user interface with a new design system



AI Resume Analyzer

Resumes in, a ranked and explained shortlist out.